



TECNOLÓGICO NACIONAL DE MÉXICO CAMPUS CIUDAD JUÁREZ
INSTITUTO TECNOLÓGICO DE CIUDAD JUÁREZ

Programación Web

Unidad #3

Reyna Morisco Luis David

22111443

Ingeniería en Sistemas Computacionales

Ejercicio 3.2: Compendio de ejercicios — creación de páginas en React

Contreras Masse Roberto

Ciudad Juárez, Chihuahua, México, a 5 de octubre de 2026

Índice

Introducción 3

1. Entorno de trabajo y organización del proyecto 4

1.1 Entorno de trabajo 4

1.2 Estructura de carpetas y estilos 5

1.3 Comandos del proyecto 6

2. Ejercicio 1 — Tu primer componente y JSX 7

2.1 Objetivo 7

2.2 Desarrollo 7

2.3 Cumplimiento de requisitos 7

2.4 Evidencias 7

2.5 Verificación 8

3. Ejercicio 2 — Props: de un componente fijo a uno reutilizable 9

3.1 Objetivo 9

3.2 Desarrollo 9

3.3 Cumplimiento de requisitos 9

3.4 Evidencias 10

3.5 Verificación 11

4. Ejercicio 3 — Composición: una página armada de varios componentes 12

4.1 Objetivo 12

4.2 Desarrollo 12

4.3 Cumplimiento de requisitos 12

4.4 Evidencias 13

4.5 Verificación 14

5. Ejercicio 4 — Listas: renderizar una colección de tarjetas 15

5.1 Objetivo 15
5.2 Relación con el proyecto integrador 15
5.3 Desarrollo 15
5.4 Cumplimiento de requisitos 16
5.5 Evidencias 16
5.6 Verificación 18
6. Ejercicio 5 — Estado con useState: una página que reacciona a la interacción 19
6.1 Objetivo 19
6.2 Desarrollo 19
6.3 Cumplimiento de requisitos 19
6.4 Evidencias 20
6.5 Verificación 21
7. Ejercicio 6 — Reto integrador: una página completa 22
7.1 Objetivo 22
7.2 Desarrollo 22
7.3 Cumplimiento de requisitos 22
7.4 Evidencias 23
7.5 Criterios de autoevaluación 25
8. Reutilización de componentes 27
9. Dificultades encontradas y soluciones 28
10. Verificación general del proyecto 29
Conclusiones 30

Introducción

El Ejercicio 3.2 es un compendio de seis ejercicios progresivos para practicar la construcción de interfaces con React. Avanza desde un componente con datos fijos hasta una página completa que combina composición de componentes, props, listas y estado. El compendio no ofrece una solución única; cada ejercicio plantea un objetivo, una serie de requisitos y unas preguntas de verificación, y la forma de estructurar el código queda a criterio de quien lo resuelve.

Este documento describe cómo se resolvió cada ejercicio: qué componentes se crearon, cómo se cumplió cada requisito, qué se obtuvo al responder las preguntas de verificación y qué capturas de código y de resultado respaldan el trabajo. El código fuente se encuentra en el repositorio.

1. Entorno de trabajo y organización del proyecto

1.1 Entorno de trabajo

El proyecto se ejecutó en la carpeta D:\Ejercicio_3.2 y sus dependencias se instalaron con npm install, que agregó 27 paquetes. La siguiente tabla reúne los datos del equipo.

Tabla 1. *Datos del entorno de trabajo*

Dato	Valor
Sistema operativo	Windows 11
Versión de Node.js	24.20.0
Versión de npm	11.19.0
Editor de código	Visual Studio Code
Terminal utilizada	Terminal integrada de VS Code (PowerShell)
Ubicación del proyecto	D:\Ejercicio_3.2

El proyecto es una aplicación de Vite con la plantilla de React y TypeScript. Las versiones de sus herramientas son las que fija el archivo package-lock.json, que npm install respeta para instalar exactamente las mismas en cualquier equipo:

Tabla 2. *Herramientas del proyecto y versiones instaladas*

Herramienta	Versión	Uso en el proyecto
Vite	8.3.2	Servidor de desarrollo con recarga automática y compilación para producción.
React y React DOM	19.3.0	Biblioteca para construir la interfaz con componentes.
TypeScript	6.0.3	Tipado de props, datos y estado; detecta errores antes de ejecutar.
Oxlint	1.86.0	Análisis estático del código con reglas de React y TypeScript.
Herramientas de desarrollador del navegador	—	Revisar la consola y la estructura del DOM generada.

1.2 Estructura de carpetas y estilos

Siguiendo la convención del compendio, dentro de src/ se creó la carpeta ejercicios/ con una subcarpeta por ejercicio. Así cada ejercicio conserva su propio código y ninguno sobrescribe al anterior. Cuando un ejercicio reutiliza un componente de otro, lo importa desde su carpeta original en lugar de copiarlo.

```
src/
  ejercicios/
    ejercicio-1/  TarjetaPersonal.tsx
    ejercicio-2/  TarjetaUsuario.tsx, Ejercicio2.tsx
    ejercicio-3/  Encabezado.tsx, PiePagina.tsx,
                  PaginaPerfil.tsx
    ejercicio-4/  datos.ts, ListaDeTarjetas.tsx
    ejercicio-5/  Buscador.tsx, ListaCompacta.tsx
    ejercicio-6/  orden.ts, SelectorOrden.tsx,
                  PaginaCatalogo.tsx
  App.tsx         muestra cada ejercicio en su sección
  index.css       estilos generales y colores
  main.tsx        punto de entrada de la aplicación
```

Cada componente tiene, además, su propia hoja de estilos CSS con el mismo nombre. En lugar de importar temporalmente un solo ejercicio a la vez, App.tsx muestra los seis ejercicios en secciones consecutivas de la misma página, cada una con su título. Así, con el servidor de desarrollo corriendo, se pueden revisar todos los resultados juntos.

Los estilos se aplican únicamente con className, sin estilos en línea. Como una hoja CSS normal es global, cada componente usa nombres de clase propios con el formato bloque__elemento, por ejemplo tarjeta-usuario__nombre, para que las reglas de un componente no se mezclen con las de otro. Los colores se definen una sola vez como variables en index.css.

1.3 Comandos del proyecto

Tabla 3. *Comandos disponibles*

Comando	Qué hace
`npm install`	Instala las dependencias del proyecto.
`npm run dev`	Inicia el servidor de desarrollo en http://localhost:5173 y actualiza la página cada vez que se guarda un cambio. Mientras corre, ocupa la terminal.
`npm run build`	Verifica los tipos con TypeScript (tsc) y genera la versión de producción.
`npm run lint`	Analiza el código con Oxlint.

Como npm run dev deja el servidor corriendo y ocupa su terminal, npm run build y npm run lint se ejecutan en una segunda terminal (botón “+” del panel de la terminal de VS Code) o después de detener el servidor con Ctrl + C.

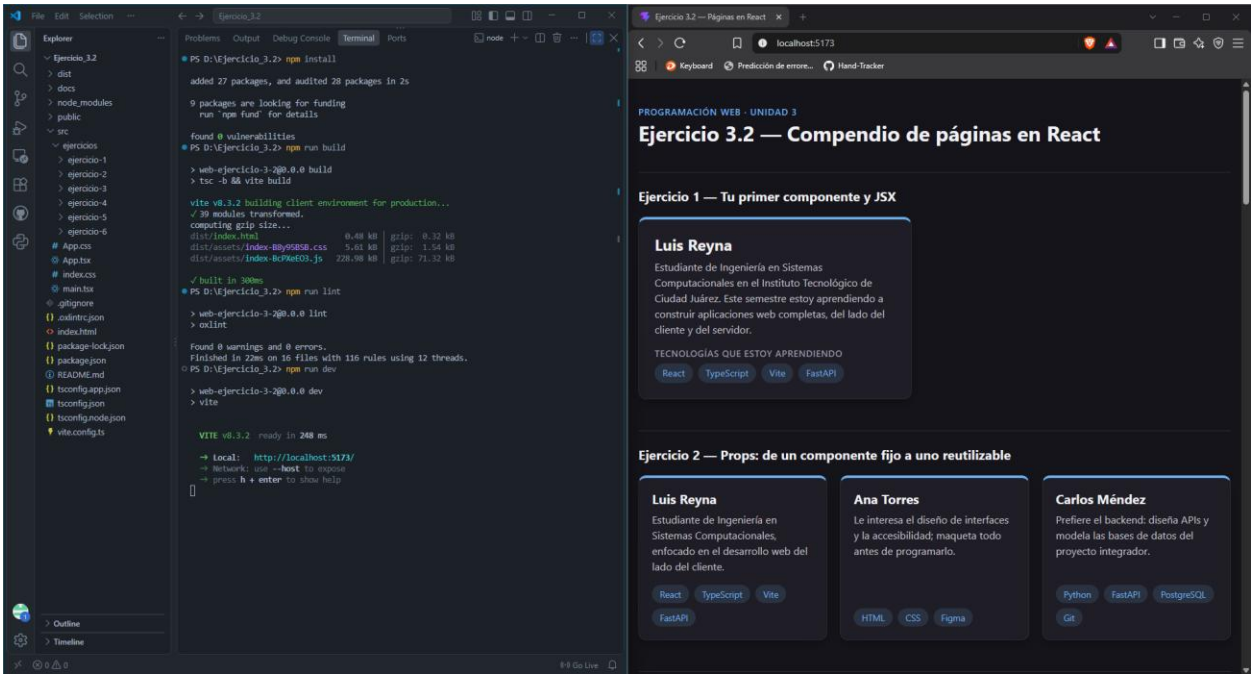


Figura 1. *Estructura de carpetas del proyecto y servidor de desarrollo en ejecución.*

2. Ejercicio 1 — Tu primer componente y JSX

2.1 Objetivo

Escribir un componente funcional que regrese JSX, aplicando una sintaxis básica: un componente es una función que devuelve una descripción de la interfaz.

2.2 Desarrollo

En ejercicio-1/TarjetaPersonal.tsx se creó el componente TarjetaPersonal, una función sin parámetros que devuelve un único <article> con el nombre, una breve descripción y una lista con cuatro tecnologías. Los datos están escritos directamente en el JSX y los estilos se aplican con className desde TarjetaPersonal.css. El componente se exporta con export default para importarlo en App.tsx.

2.3 Cumplimiento de requisitos

Tabla 4. *Requisitos del compendio y cómo se cumplieron*

Requisito del compendio	Cómo se cumplió
Componente TarjetaPersonal con un encabezado con el nombre, un párrafo de descripción y una lista no ordenada con al menos tres tecnologías.	El componente muestra un <h2> con el nombre, un <p> con la descripción y un con cuatro tecnologías.
Regresar un único elemento raíz que envuelva todo el contenido.	Todo el contenido está dentro de un solo <article className="tarjeta-personal">.
Dar estilo básico solo con className y una hoja de estilos CSS normal.	Las clases se definen en TarjetaPersonal.css; no se usa style.

2.4 Evidencias

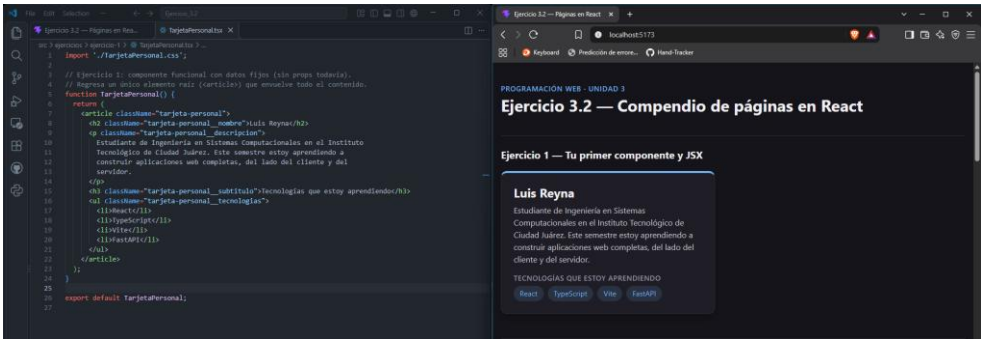


Figura 2. *Componente TarjetaPersonal y su resultado en el navegador.*

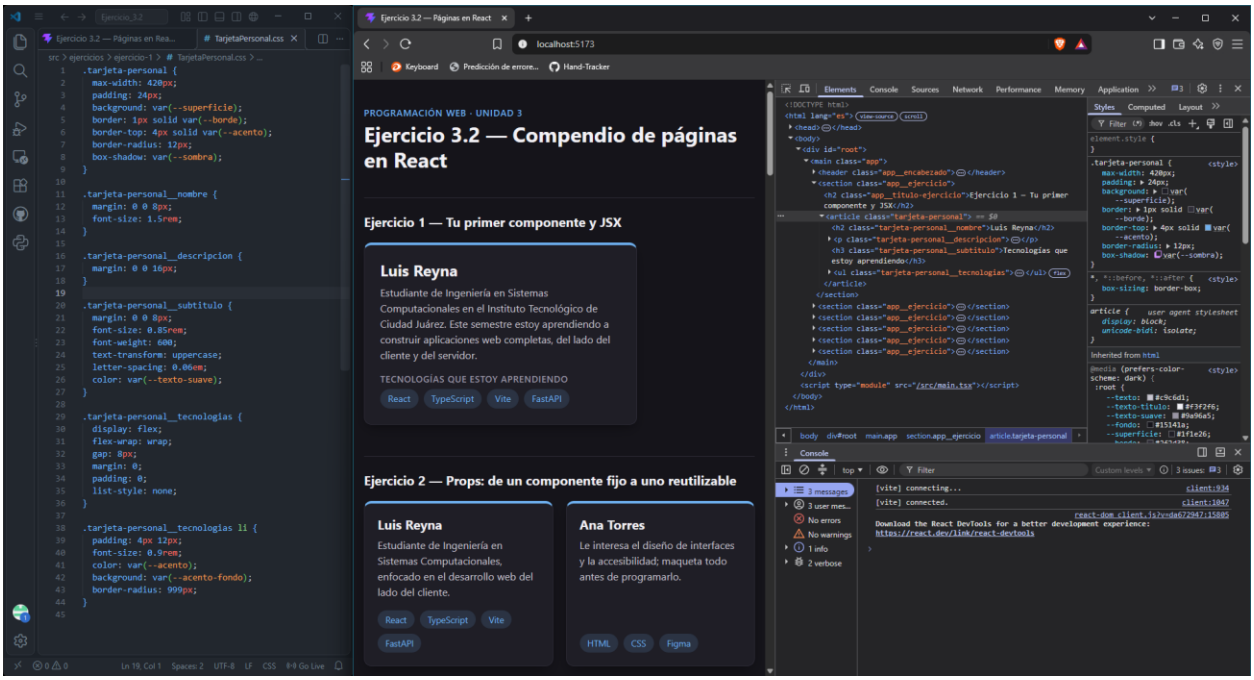


Figura 3. *Hoja de estilos del componente y estructura del DOM con un solo elemento raíz.*

2.5 Verificación

Tabla 5. *Preguntas de verificación del Ejercicio 1*

Pregunta del compendio	Respuesta y evidencia
¿El componente regresa exactamente un elemento raíz, sin errores de JSX en la consola del navegador?	Sí. La función devuelve un solo <article> que envuelve a los demás elementos. En las herramientas de desarrollador se ve un único nodo article.tarjeta-personal, y la consola no muestra errores.
¿El archivo exporta el componente con export default, de modo que se pueda importar desde App.tsx?	Sí. La última línea es export default TarjetaPersonal; y App.tsx lo importa con import TarjetaPersonal from './ejercicios/ejercicio-1/TarjetaPersonal';.
¿Se usó className en lugar del atributo class de HTML?	Sí. JSX se transforma en llamadas de JavaScript y class es una palabra reservada de ese lenguaje; por eso React usa className. En el DOM final el navegador muestra el atributo class normal (Figura 3).

3. Ejercicio 2 — Props: de un componente fijo a uno reutilizable

3.1 Objetivo

Transformar un componente de datos fijos en uno reutilizable mediante props, en donde un componente recibe datos de entrada y describe qué interfaz debe renderizarse a partir de ellos.

3.2 Desarrollo

TarjetaPersonal se convirtió en el componente genérico TarjetaUsuario, que recibe nombre, descripcion y tecnologias por props. Sus tipos se declaran en la interfaz TarjetaUsuarioProps y las props se desestructuran en los parámetros de la función. El arreglo de tecnologías se muestra con map(), usando cada tecnología como key. El contenedor Ejercicio2 renderiza tres instancias con datos distintos: la tarjeta propia y dos perfiles de ejemplo.

3.3 Cumplimiento de requisitos

Tabla 6. *Requisitos del compendio y cómo se cumplieron*

Requisito del compendio	Cómo se cumplió
Convertir TarjetaPersonal en el componente genérico TarjetaUsuario, que reciba por props un nombre, una descripción y un arreglo de tecnologías.	TarjetaUsuario recibe las tres props y no tiene datos escritos en su código.
Declarar el tipo de las props con una interfaz de TypeScript.	interface TarjetaUsuarioProps
Renderizar al menos tres instancias de TarjetaUsuario con datos distintos.	Ejercicio2.tsx renderiza tres tarjetas con nombres, descripciones y tecnologías diferentes.
Pregunta del compendio: ¿cómo renderizar el arreglo de tecnologías?	Con tecnologias.map() y una key en cada .

3.4 Evidencias

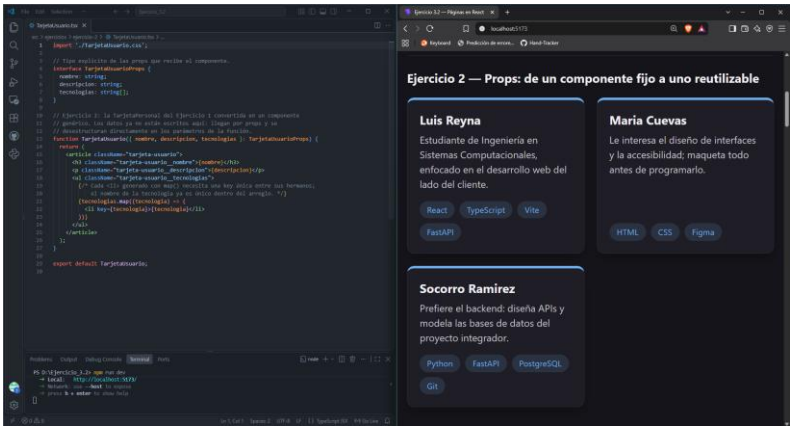


Figura 4. *Componente TarjetaUsuario con props tipadas.*

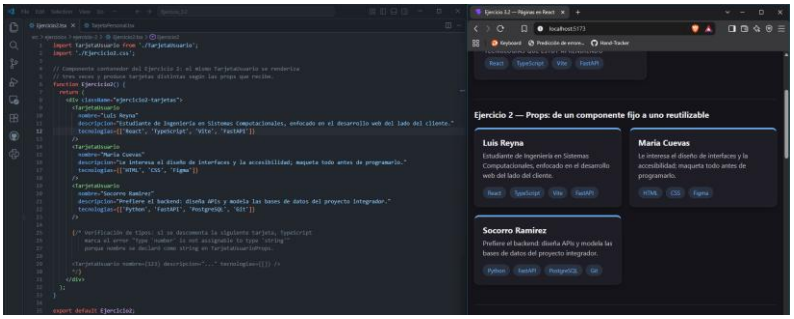


Figura 5. *Tres instancias del mismo componente con datos distintos.*

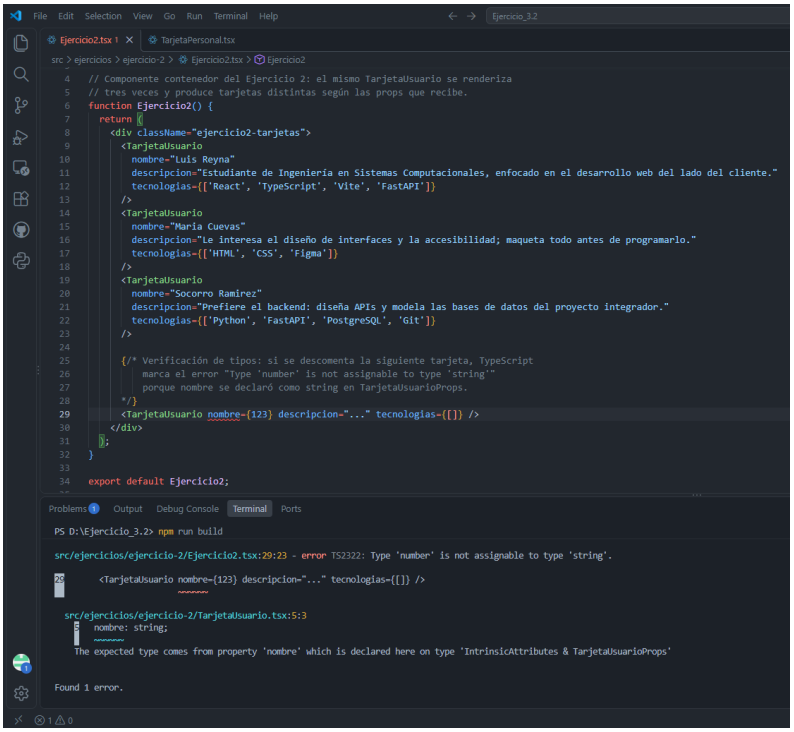


Figura 6. *TypeScript rechaza un número donde se espera una cadena.*

3.5 Verificación

Tabla 7. *Preguntas de verificación del Ejercicio 2*

Pregunta del compendio	Respuesta y evidencia
¿El componente usa desestructuración de props en los parámetros de la función?	Sí: function TarjetaUsuario({ nombre, descripcion, tecnologias }: TarjetaUsuarioProps) (Figura 4).
¿Cambiar los valores que se le pasan a cada instancia cambia lo que se muestra, sin tocar el código del componente?	Sí. Las tres tarjetas de la Figura 5 salen del mismo componente; lo único distinto son las props escritas en Ejercicio2.tsx. Al editar, por ejemplo, la descripción de una instancia y guardar, Vite actualiza la página y solo cambia esa tarjeta.
¿TypeScript marca un error si, a propósito, se pasa un número donde se espera una cadena?	Sí. Con nombre={123}, el editor subraya la prop y npm run build termina con “error TS2322: Type 'number' is not assignable to type 'string'” (Figura 6). El servidor de desarrollo de Vite no verifica tipos, así que la página puede seguir mostrándose; el error lo detectan el editor y tsc. La línea de prueba quedó comentada en Ejercicio2.tsx como referencia.

4. Ejercicio 3 — Composición: una página armada de varios componentes

4.1 Objetivo

Practicar el principio de composición donde se construye una interfaz completa anidando componentes pequeños dentro de componentes más grandes, en lugar de escribir una sola función con todo el marcado.

4.2 Desarrollo

La página PaginaPerfil se compone de tres componentes: Encabezado (título y subtítulo por props), TarjetaUsuario (reutilizado del Ejercicio 2) y PiePagina (texto de cierre por prop). PaginaPerfil no tiene marcado propio: devuelve a sus hijos dentro de un Fragmento y les pasa todos sus datos como props. En App.tsx se coloca dentro de un marco que la presenta como una página independiente.

4.3 Cumplimiento de requisitos

Tabla 8. *Requisitos del compendio y cómo se cumplieron*

Requisito del compendio	Cómo se cumplió
PaginaPerfil compuesta de al menos tres componentes distintos: un Encabezado, el TarjetaUsuario del Ejercicio 2 (reutilizado, no reescrito) y un PiePagina.	Los tres componentes están en archivos separados; TarjetaUsuario se importa desde la carpeta del Ejercicio 2.
PaginaPerfil no debe tener prácticamente marcado propio; solo ensambla a sus hijos en orden.	Devuelve un Fragmento con los tres componentes.
Al menos un dato debe fluir desde PaginaPerfil hacia sus hijos como prop.	Fluyen todos: titulo y subtitulo hacia Encabezado, los datos de la tarjeta hacia TarjetaUsuario y texto hacia PiePagina.

4.4 Evidencias

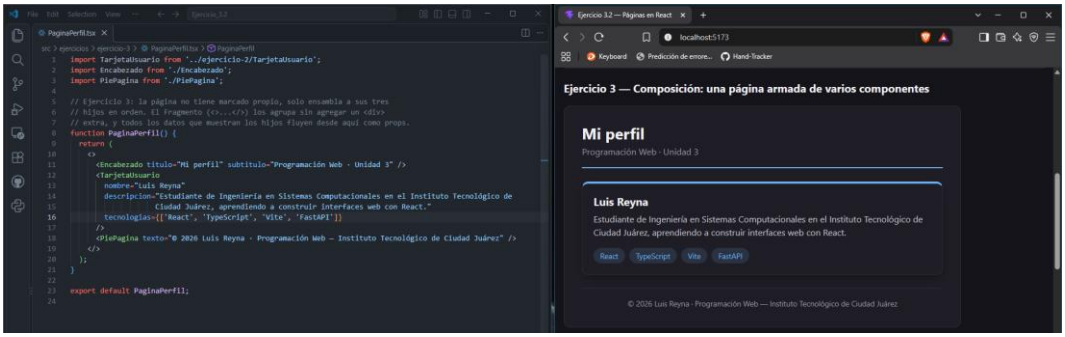


Figura 7. *Página PaginaPerfil compuesta por tres componentes.*

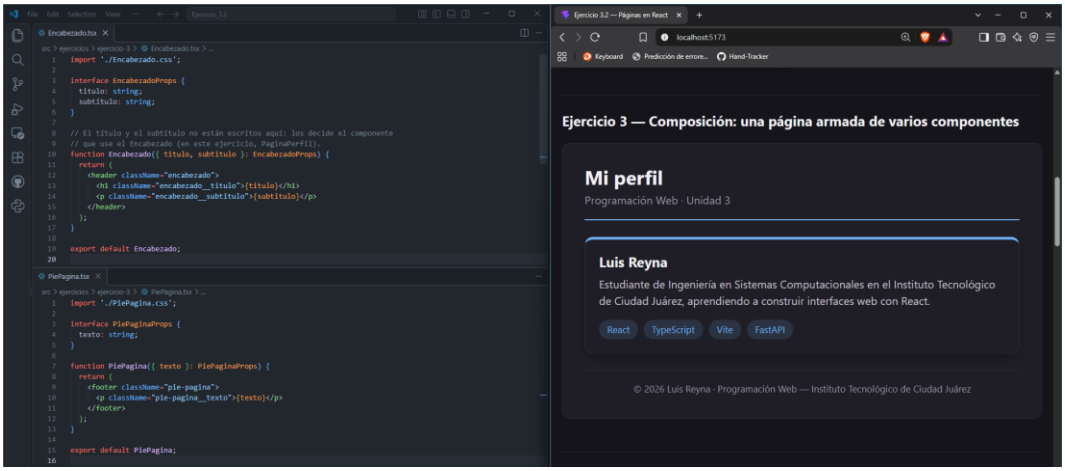


Figura 8. *Componentes Encabezado y PiePagina, configurables por props.*

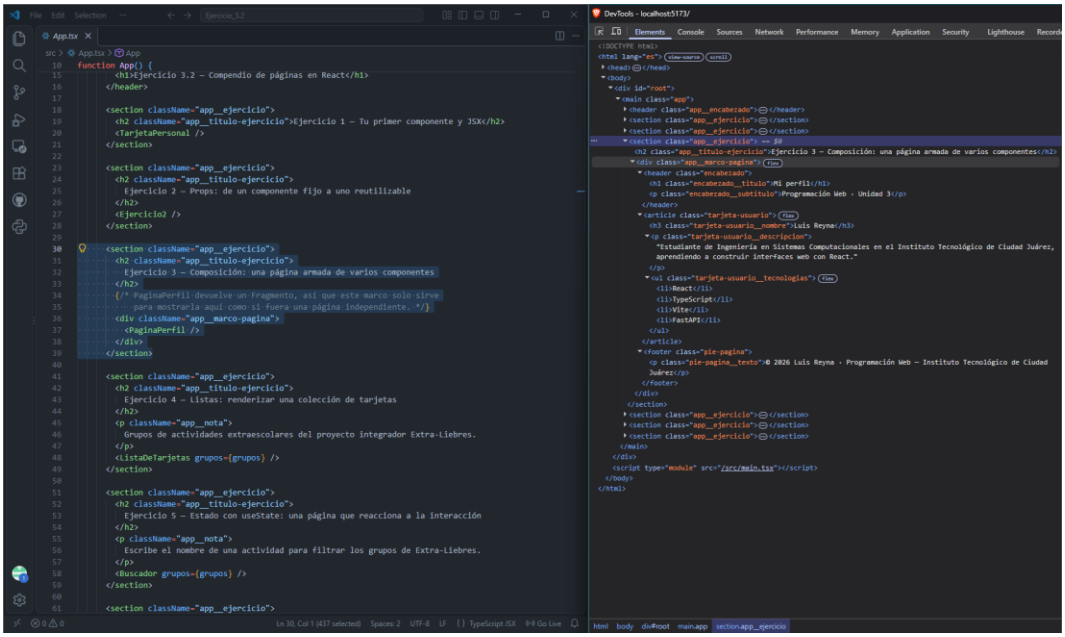


Figura 9. *El Fragmento no agrega elementos al DOM.*

4.5 Verificación

Tabla 9. *Preguntas de verificación del Ejercicio 3*

Pregunta del compendio	Respuesta y evidencia
¿Se puede identificar, viendo solo el código de PaginaPerfil, qué tres piezas componen la página y en qué orden aparecen?	Sí. El código de PaginaPerfil contiene solo tres etiquetas, en el mismo orden en que aparecen en pantalla: Encabezado, TarjetaUsuario y PiePagina (Figura 7).
Si se cambia el texto de titulo en la llamada a Encabezado, ¿el subtítulo también es prop o quedó fijo dentro del componente?	Ambos son props obligatorias de EncabezadoProps. Al cambiar titulo o subtitulo en PaginaPerfil, cambia el texto correspondiente sin tocar Encabezado.tsx (Figura 8). Además, si se usa <Encabezado> sin subtitulo, TypeScript marca el error TS2741 “Property 'subtitulo' is missing”, lo que confirma que no quedó fijo dentro del componente.

5. Ejercicio 4 — Listas: renderizar una colección de tarjetas

5.1 Objetivo

Practicar el renderizado de listas con map() y la prop key en un escenario con varios elementos del mismo tipo, el patrón que se repetirá en el proyecto integrador para mostrar colecciones de registros.

5.2 Relación con el proyecto integrador

El compendio pide que los datos representen algo relevante para el proyecto integrador, Extra-Liebres. En ese proyecto, la entidad grupo reúne la información de una actividad extraescolar impartida en un semestre: la actividad, el semestre, el promotor, el día, el horario y el aula. Por eso se eligió representar grupos: generan tarjetas con información útil y son la base de la búsqueda por actividad que plantea el requerimiento funcional 6 del proyecto. Como todavía no hay API, los datos de las tablas relacionadas (extraescolar, semestre y promotor) se escribieron resueltos como texto.

Tabla 10. *Correspondencia entre los datos del ejercicio y el modelo de Extra-Liebres*

Campo en datos.ts	Origen en el modelo de Extra-Liebres	Ejemplo
id	idgrupo, llave primaria de grupo	101
titulo	nombreextra de la entidad extraescolar	Fútbol soccer
descripcion	Semestre (mesinicio, mestermino, anio) y promotor a cargo	Grupo del semestre agosto–diciembre 2021, a cargo de Jorge Ramírez Soto.
dia	día	Lunes
horario	horainicio y horatermino	16:00–18:00
aula	aula	Cancha de fútbol

5.3 Desarrollo

En datos.ts se definieron la interfaz Grupo y un arreglo con ocho grupos de ejemplo de Extra-Liebres; “Fútbol soccer” aparece en dos semestres para que haya títulos repetidos.

ListaDeTarjetas recibe el arreglo por props y genera una TarjetaUsuario por grupo con map(); el día, el horario y el aula se pasan como etiquetas en la prop tecnologias. Como key se usa grupo.id, porque el título se repite y el índice cambia al filtrar o reordenar la lista.

5.4 Cumplimiento de requisitos

Tabla 11. *Requisitos del compendio y cómo se cumplieron*

Requisito del compendio	Cómo se cumplió
Definir en un archivo separado un arreglo de al menos seis objetos relevantes al proyecto integrador, cada uno con identificador único, título y breve descripción.	datos.ts contiene ocho grupos de Extra-Liebres con id, titulo y descripcion, además de dia, horario y aula.
Construir ListaDeTarjetas, que reciba el arreglo como prop y renderice una tarjeta por elemento con map(), sin copiar JSX a mano.	ListaDeTarjetas recibe grupos y usa grupos.map() para generar las tarjetas.
Usar el identificador único como key, no el índice de posición.	key={grupo.id}.

5.5 Evidencias

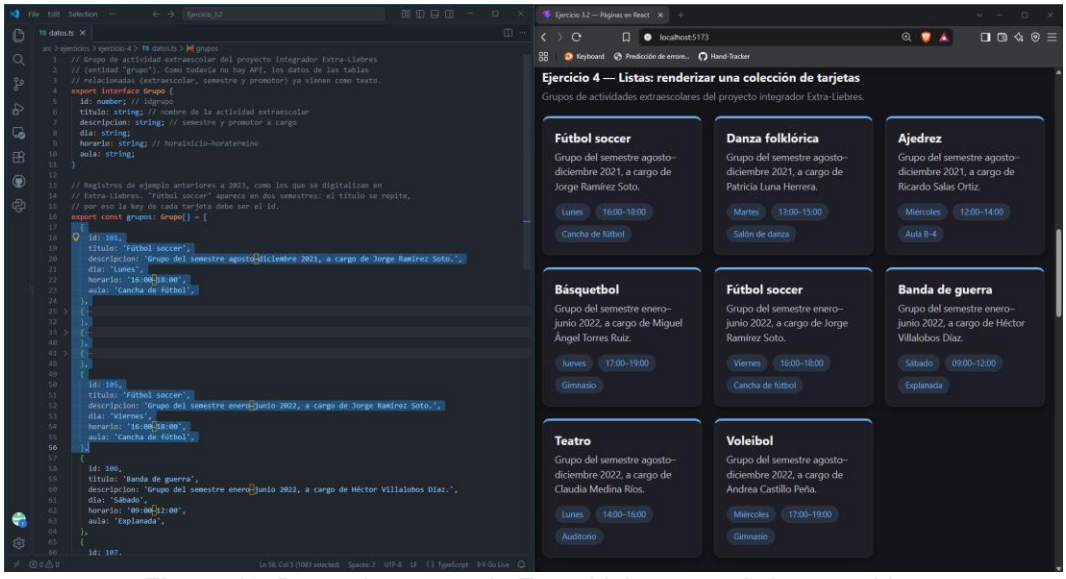


Figura 10. *Datos de grupos de Extra-Liebres con títulos repetidos.*

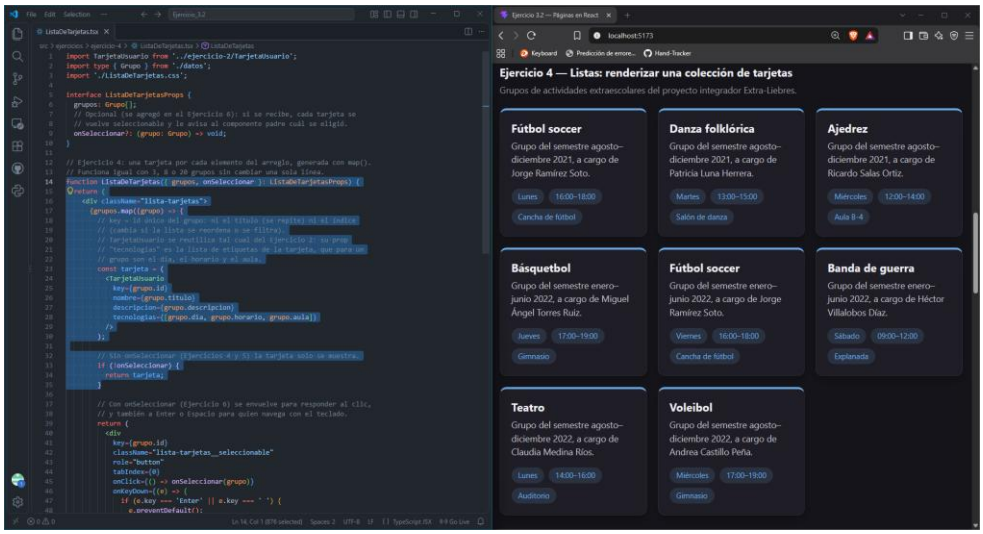


Figura 11. *ListaDeTarjetas genera una tarjeta por grupo con map().*

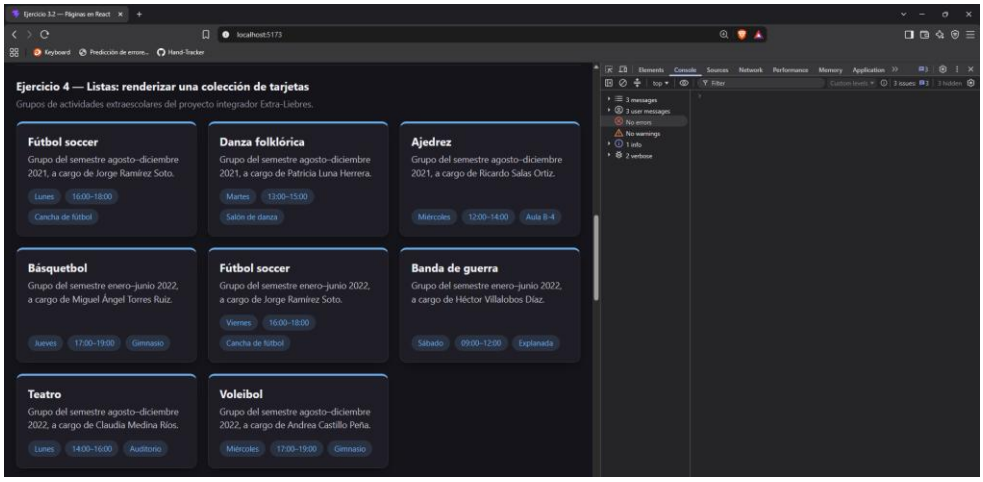


Figura 12. *La consola no muestra advertencias de key.*

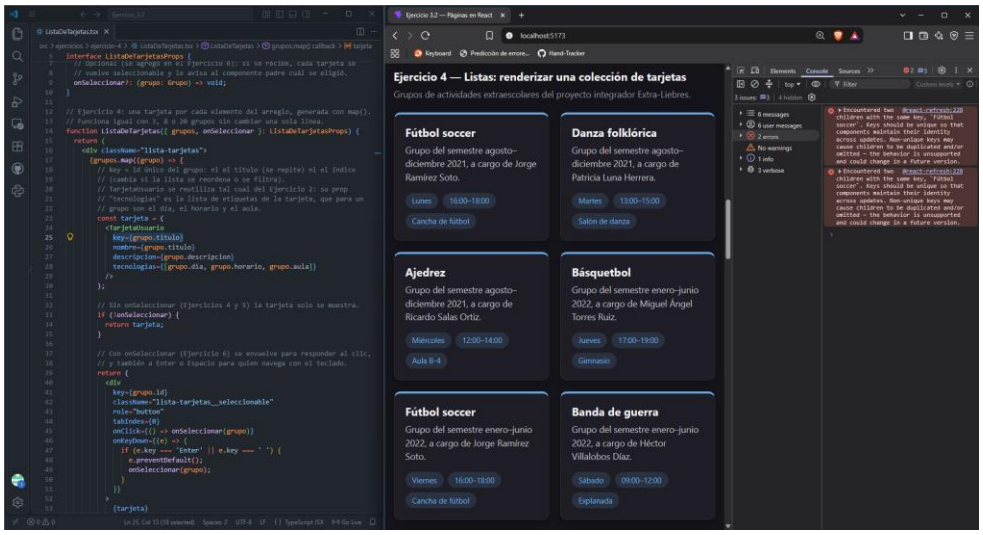


Figura 13. *Con el título como key, React detecta claves duplicadas.*

5.6 Verificación

Tabla 12. *Preguntas de verificación del Ejercicio 4*

Pregunta del compendio	Respuesta y evidencia
¿Aparece en la consola alguna advertencia sobre la prop key faltante o duplicada?	No. Con key={grupo.id} la consola no muestra advertencias (Figura 12). Como comprobación adicional, se cambió temporalmente la key por grupo.titulo y React mostró el error “Encountered two children with the same key”, provocado por los dos grupos de “Fútbol soccer” (Figura 13). Por eso la key debe ser el id.
¿ListaDeTarjetas funcionaría igual si el arreglo tuviera 3 o 20 elementos, sin cambiar su código?	Sí. El componente no conoce la cantidad de grupos: map() genera tantas tarjetas como elementos tenga el arreglo. El compendio pedía seis y se usaron ocho sin cambiar el componente; además, en los ejercicios 5 y 6 el mismo componente recibe listas filtradas de 0 a 8 elementos.

6. Ejercicio 5 — Estado con useState: una página que reacciona a la interacción

6.1 Objetivo

Incorporar el Hook useState para que la página deje de ser una fotografía estática y reaccione a lo que hace el usuario, aplicando el principio de inmutabilidad del estado.

6.2 Desarrollo

El componente Buscador maneja dos estados con useState: textoBusqueda, que guarda lo que se escribe y se actualiza en cada tecla con onChange, y vistaCompacta, un booleano que el botón invierte en cada clic. La lista se recalcula en cada render con filter() sobre el arreglo completo, sin modificarlo, y compara los títulos sin distinguir mayúsculas ni acentos. Según vistaCompacta, se muestra ListaDeTarjetas (Ejercicio 4) o ListaCompacta, una lista de solo texto. Ambos estados viven en Buscador porque ahí están los controles que los cambian y la lista que los usa.

6.3 Cumplimiento de requisitos

Tabla 13. *Requisitos del compendio y cómo se cumplieron*

Requisito del compendio	Cómo se cumplió
Componente Buscador con un <input> y, debajo, la lista de tarjetas del Ejercicio 4 filtrada según lo que se escribe.	Buscador muestra el campo y debajo ListaDeTarjetas (o ListaCompacta) con los grupos filtrados.
Guardar el valor del campo en un estado declarado con useState y actualizarlo en cada tecla con onChange.	const [textoBusqueda, setTextoBusqueda] = useState('') y onChange={(e) => setTextoBusqueda(e.target.value)}.
Recalcular la lista a partir del arreglo completo y del texto actual, mostrando los elementos cuyo título contenga el texto, sin distinguir mayúsculas de minúsculas.	grupos.filter() con normalizar(grupo.titulo).includes(busqueda).

Requisito del compendio	Cómo se cumplió
Segundo estado: botón que alterne entre la lista en tarjetas y una lista simple de solo texto, con un booleano que se invierte en cada clic.	const [vistaCompacta, setVistaCompacta] = useState(false) y setVistaCompacta((compacta) => !compacta).
No mutar en ningún punto el arreglo de datos original.	Solo se usa filter(), que devuelve un arreglo nuevo.

6.4 Evidencias

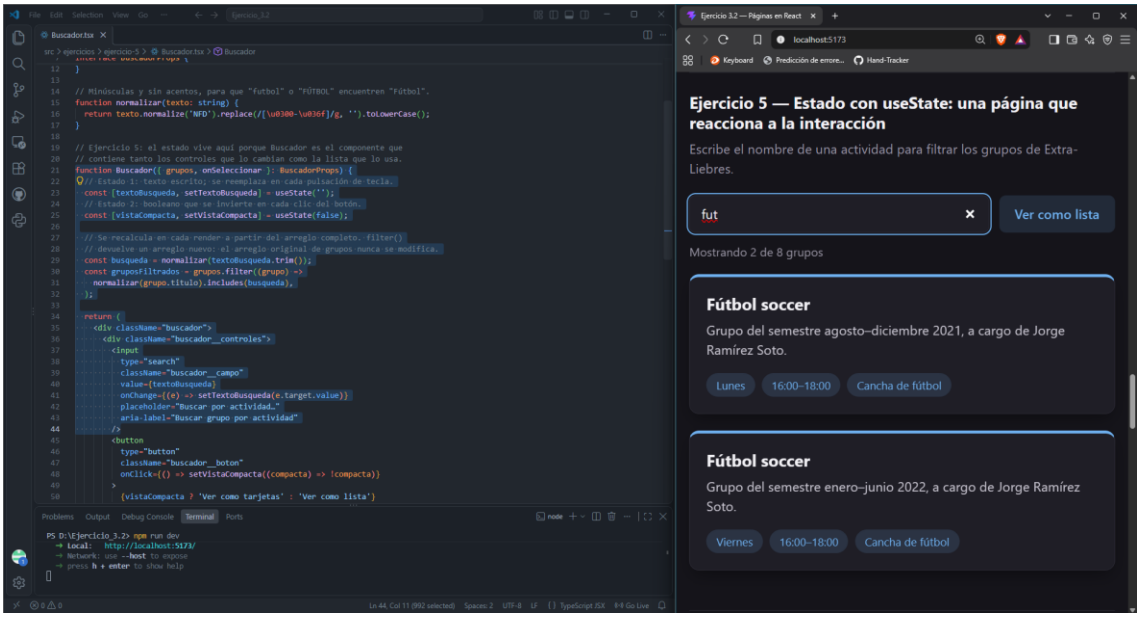


Figura 14. *Estado del texto de búsqueda y filtrado en cada tecla.*

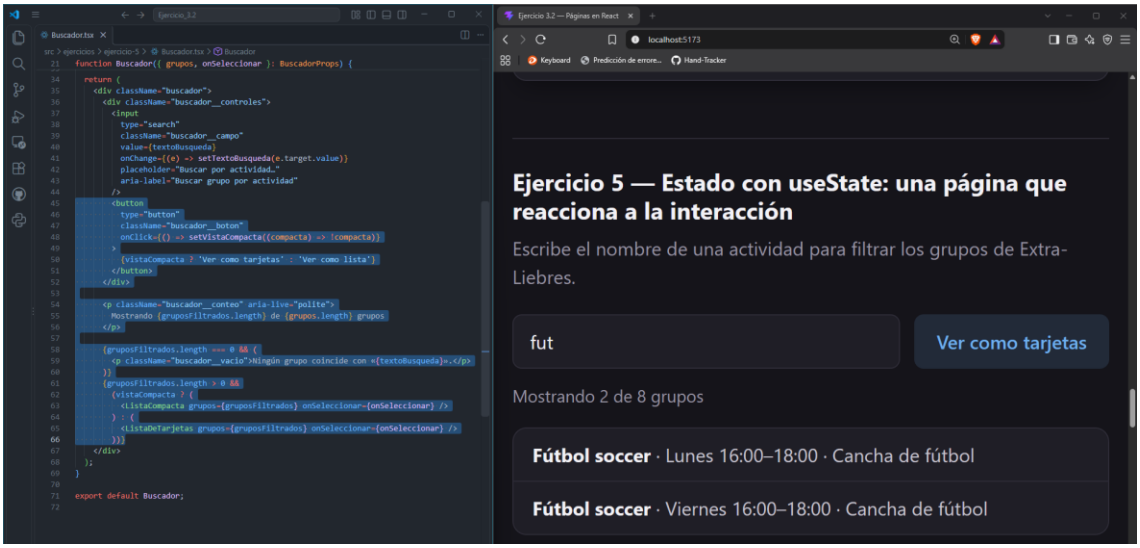


Figura 15. *El botón cambia la vista sin borrar la búsqueda.*

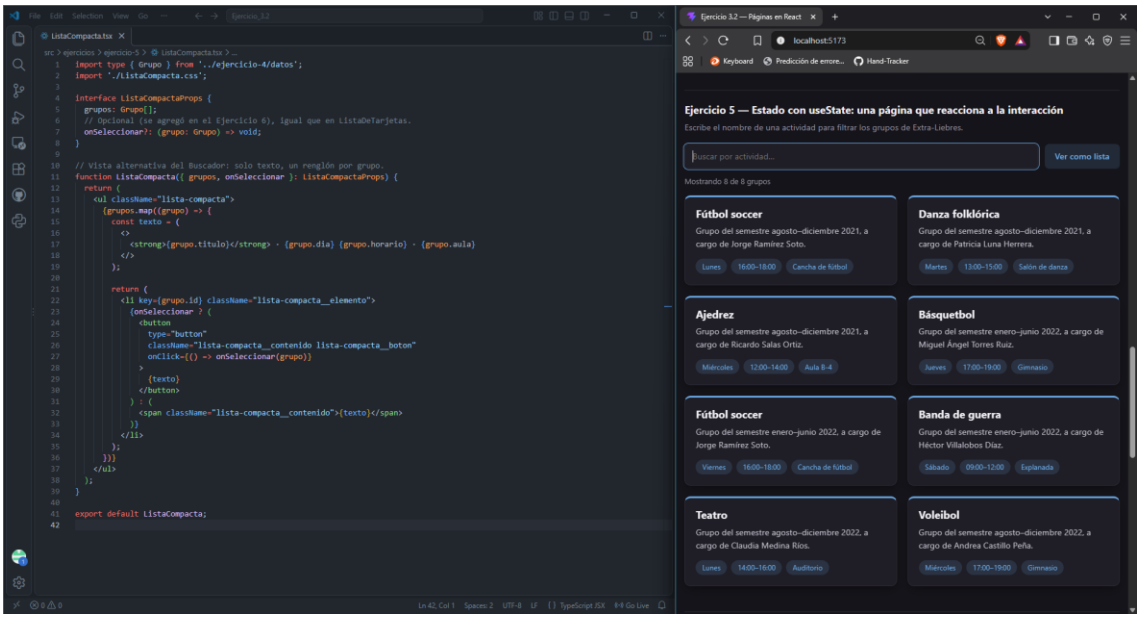


Figura 16. *Con el campo vacío vuelve a aparecer la lista completa.*

6.5 Verificación

Tabla 14. *Preguntas de verificación del Ejercicio 5*

Pregunta del compendio	Respuesta y evidencia
¿La lista se actualiza en cada tecla, sin tener que hacer clic en un botón de “buscar”?	Sí. Al escribir “f” quedan 3 grupos, con “fu” quedan 2 y con “fut” siguen 2 (Figura 14). No hay botón de búsqueda: cada tecla dispara onChange, cambia el estado y React vuelve a renderizar con la lista recalculada.
Si se borra todo el texto del campo, ¿vuelve a aparecer la lista completa?	Sí. Con el campo vacío, el texto de búsqueda es una cadena vacía, que está contenida en cualquier título; por eso filter() devuelve los ocho grupos (Figura 16).
¿El botón de cambiar vista funciona de forma independiente al campo de búsqueda?	Sí. Con “fut” escrito (dos resultados), se alternó la vista dos veces y el texto se conservó con los mismos dos grupos (Figura 15). Son dos estados separados, y cambiar uno no modifica el otro.

7. Ejercicio 6 — Reto integrador: una página completa

7.1 Objetivo

Combinar en una sola página coherente todo lo practicado en los cinco ejercicios anteriores, de la misma forma en que se hará en el proyecto integrador. El compendio establece tres requisitos no negociables: composición de componentes, al menos dos tipos distintos de estado y al menos una función pasada como prop.

7.2 Desarrollo

PaginaCatalogo ensambla Encabezado y PiePagina (Ejercicio 3), Buscador (Ejercicio 5) y el nuevo SelectorOrden. El tercer estado, orden, vive en PaginaCatalogo porque lo cambia SelectorOrden y lo usa Buscador, y PaginaCatalogo es el padre común de ambos; la función ordenarGrupos() ordena una copia del arreglo, sin modificar el original. Se pasan dos funciones como prop: onCambiar, con la que SelectorOrden avisa qué orden se eligió, y onSeleccionar, que llega a las tarjetas a través de Buscador y escribe en la consola el grupo seleccionado. Esa prop es opcional, así que los ejercicios 4 y 5 siguen funcionando igual.

7.3 Cumplimiento de requisitos

Tabla 15. *Requisitos del compendio y cómo se cumplieron*

Requisito del compendio	Cómo se cumplió
Página PaginaCatalogo con un Encabezado reutilizado del Ejercicio 3, un Buscador con filtrado y estado como el del Ejercicio 5 y un PiePagina.	Los tres componentes se importan desde sus carpetas originales; además se agregó SelectorOrden.
Tercer elemento de estado: un selector que permita ordenar la lista, con el criterio guardado en una variable de estado.	SelectorOrden (<select>) y const [orden, setOrden] = useState<Orden>('original') en PaginaCatalogo.
Al menos un componente debe recibir una función como prop; la función, definida en el padre, muestra en la consola qué elemento se seleccionó.	ListaDeTarjetas y ListaCompacta reciben onSeleccionar (la función mostrarSeleccion); SelectorOrden recibe onCambiar.

Tabla 16. *Requisitos no negociables del reto*

Requisito no negociable	Cómo se cumple
Composición de componentes	PaginaCatalogo ensambla Encabezado, SelectorOrden, Buscador y PiePagina; Buscador contiene ListaDeTarjetas, que a su vez contiene TarjetaUsuario.
Al menos dos tipos distintos de estado	Texto que se reemplaza (textoBusqueda), booleano que se invierte (vistaCompacta) y valor elegido de una lista de opciones (orden).
Al menos una función pasada como prop	onSeleccionar (de PaginaCatalogo a las listas, a través de Buscador) y onCambiar (de PaginaCatalogo a SelectorOrden).

7.4 Evidencias

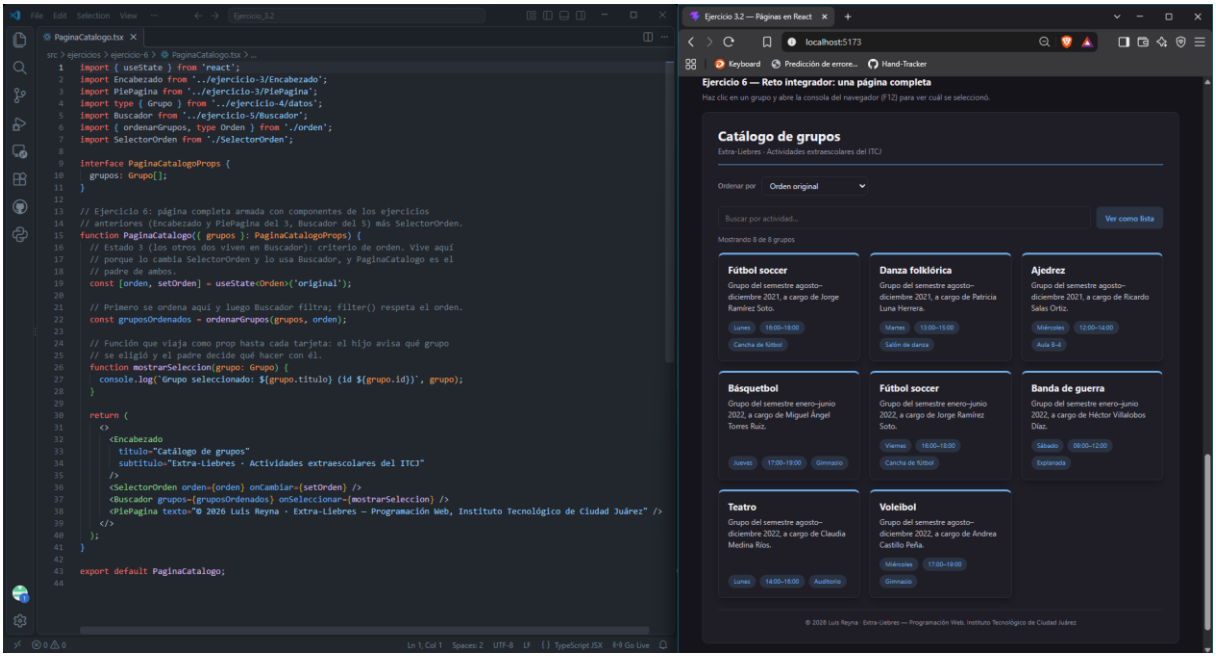


Figura 17. *Página PaginaCatalogo armada con componentes reutilizados.*

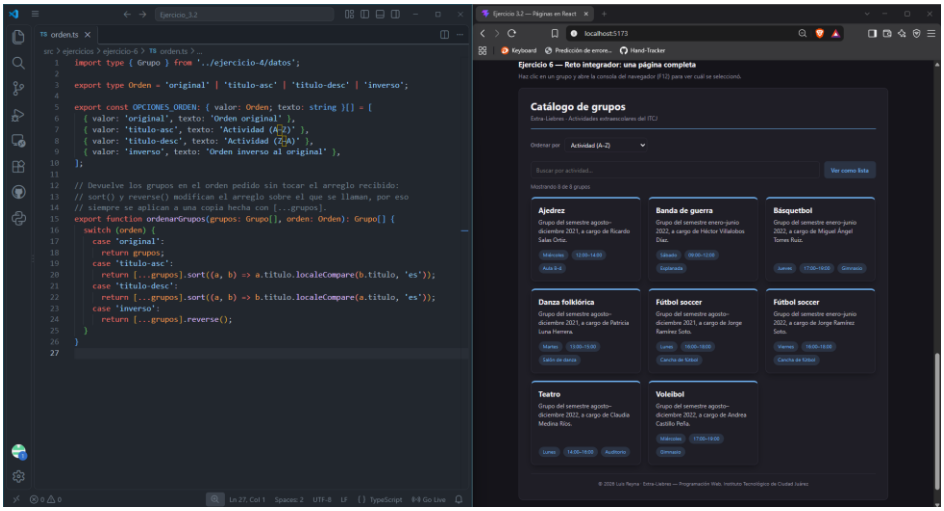


Figura 18. *Tercer estado: criterio de orden.*

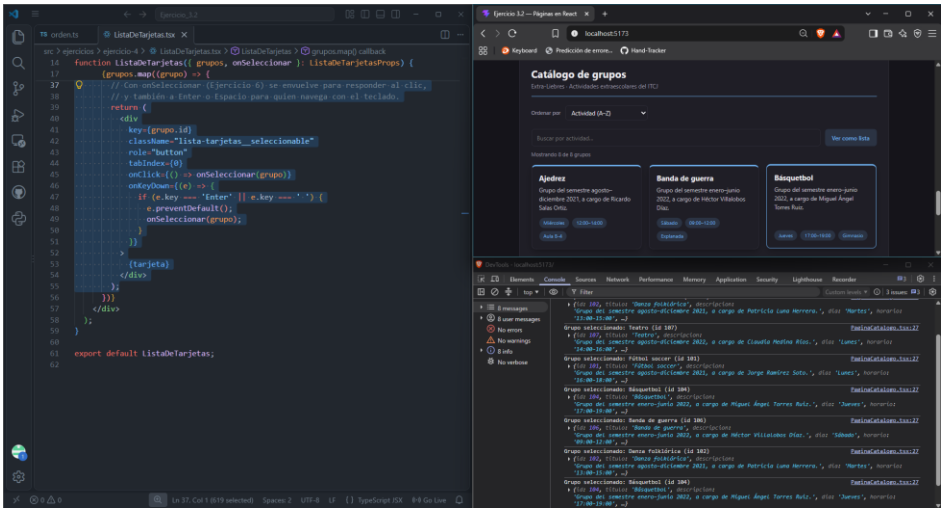


Figura 19. *La función recibida como prop avisa al padre qué grupo se seleccionó.*

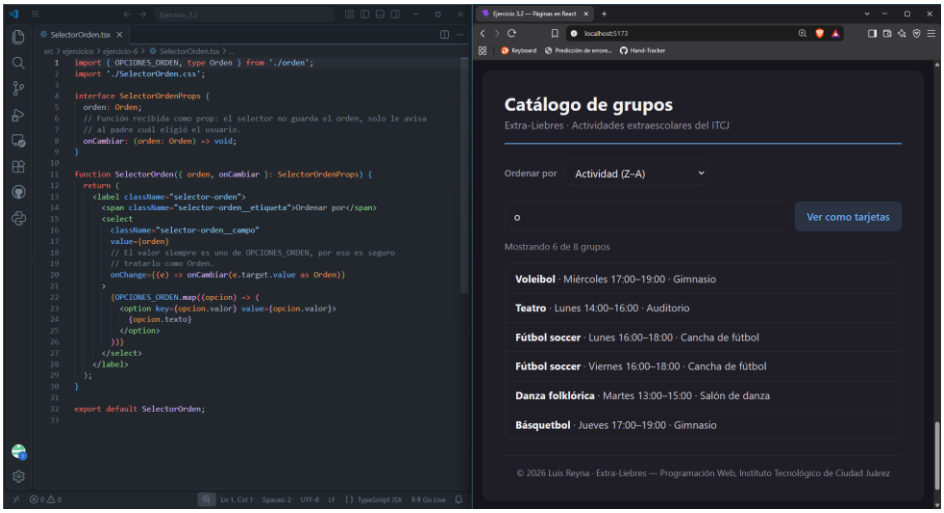


Figura 20. *Los tres estados de la página funcionan al mismo tiempo.*

7.5 Criterios de autoevaluación

Tabla 17. *Criterios de autoevaluación del Ejercicio 6*

Pregunta del compendio	Respuesta y evidencia
¿La página completa funciona sin errores en la consola del navegador?	Sí. Al buscar, ordenar, cambiar de vista y seleccionar grupos, la consola solo muestra los mensajes de mostrarSeleccion; no hay advertencias ni errores (Figura 19). Además, npm run build y npm run lint terminan sin errores (Figura 21).
¿Cada componente tiene una sola responsabilidad clara?	Sí. Cada componente hace una sola cosa, como resume la tabla de responsabilidades que sigue. La lógica de orden se separó en orden.ts para que SelectorOrden solo muestre las opciones.
¿Se puede explicar en una frase qué hace cada estado y por qué vive en el componente donde se declaró?	Sí; se explica en la tabla de estados que sigue.
¿Se reutilizaron componentes de ejercicios anteriores en lugar de escribirlos desde cero?	Sí. Encabezado y PiePagina (Ej. 3), ListaDeTarjetas (Ej. 4), Buscador y ListaCompacta (Ej. 5) y TarjetaUsuario (Ej. 2). Solo SelectorOrden y orden.ts son nuevos.

Tabla 18. *Responsabilidad de cada componente de la página*

Componente	Responsabilidad
PaginaCatalogo	Ensamblar la página, guardar el criterio de orden y decidir qué hacer cuando se selecciona un grupo.
Encabezado	Mostrar el título y el subtítulo de la página.
SelectorOrden	Mostrar las opciones de orden y avisar cuál se eligió.
Buscador	Guardar el texto de búsqueda y la vista, filtrar los grupos y mostrar la lista correspondiente.
ListaDeTarjetas	Generar una tarjeta por grupo.
ListaCompacta	Generar un renglón de texto por grupo.
TarjetaUsuario	Mostrar una tarjeta con nombre, descripción y etiquetas.
PiePagina	Mostrar el texto de cierre.
ordenarGrupos	Ordenar los grupos sin modificar el arreglo original.

Tabla 19. *Estados de la página*

Estado	Dónde vive	Qué hace	Por qué vive ahí
textoBusqueda	Buscador	Guarda lo que el usuario escribe para filtrar los grupos por actividad.	Buscador contiene el campo que lo cambia y la lista que lo usa.
vistaCompacta	Buscador	Indica si la lista se muestra en tarjetas o como texto.	Buscador contiene el botón que lo invierte y las dos vistas.
orden	PaginaCatalogo	Guarda el criterio con el que se ordenan los grupos.	Lo cambia SelectorOrden y lo usa Buscador; PaginaCatalogo es el padre común de ambos.

8. Reutilización de componentes

Los ejercicios se construyeron unos sobre otros: cada componente se escribió una sola vez y los ejercicios posteriores lo importan desde su carpeta original.

Tabla 20. *Componentes reutilizados entre ejercicios*

Componente	Creado en	Reutilizado en
TarjetaUsuario	Ejercicio 2	Ejercicio 3 (directamente) y ejercicios 4, 5 y 6 (dentro de ListaDeTarjetas).
Encabezado	Ejercicio 3	Ejercicio 6.
PiePagina	Ejercicio 3	Ejercicio 6.
datos.ts (grupos)	Ejercicio 4	Ejercicios 5 y 6.
ListaDeTarjetas	Ejercicio 4	Ejercicios 5 y 6 (dentro de Buscador).
Buscador	Ejercicio 5	Ejercicio 6.
ListaCompacta	Ejercicio 5	Ejercicio 6 (dentro de Buscador).

TarjetaPersonal (Ejercicio 1) no se reutiliza directamente: como lo indica el Ejercicio 2, se transformó en el componente genérico TarjetaUsuario, y TarjetaPersonal se conserva para mostrar el resultado del primer ejercicio.

9. Dificultades encontradas y soluciones

Tabla 21. *Dificultades y soluciones*

Dificultad	Solución
Las hojas CSS normales son globales: si dos componentes usan la misma clase (por ejemplo, tarjeta), sus reglas se mezclan.	Cada componente usa nombres de clase propios con el formato bloque__elemento, como tarjeta-personal__nombre y tarjeta-usuario__nombre.
El título de un grupo se repite (“Fútbol soccer” en dos semestres), así que no sirve como key.	Se usó el id del grupo, que es único. Se comprobó que con el título React marca el error de clave duplicada (Figura 13).
Al buscar “futbol” sin acento no aparecía “Fútbol soccer”.	Se agregó la función normalizar(), que quita los acentos y convierte a minúsculas antes de comparar.
La función onSeleccionar debía llegar desde PaginaCatalogo hasta las tarjetas, que están dentro de Buscador, sin cambiar los ejercicios 4 y 5.	Se agregó como prop opcional en Buscador, ListaDeTarjetas y ListaCompacta; si no se recibe, los componentes se comportan como antes.
Una tarjeta con onClick solo responde al mouse.	El envoltorio de cada tarjeta seleccionable tiene role="button" y tabIndex={0}, y responde a Enter y Espacio.
Al escribir npm run build y npm run lint en la terminal donde corría npm run dev, los comandos no se ejecutaban.	El servidor de desarrollo ocupa su terminal; los otros comandos se ejecutaron en una segunda terminal de VS Code.
El servidor de desarrollo de Vite no se detiene por errores de tipos, así que un error de TypeScript puede pasar desapercibido en el navegador.	Los tipos se verifican con los avisos del editor y con npm run build, que ejecuta tsc antes de compilar.

10. Verificación general del proyecto

Además de las preguntas de cada ejercicio, se verificó el proyecto completo: npm run build, que incluye la verificación de tipos con tsc, y npm run lint terminan sin errores; la consola del navegador no muestra advertencias; y la página se revisó en pantalla de escritorio, en ancho de teléfono y con los temas claro y oscuro del sistema, sin desplazamiento horizontal.

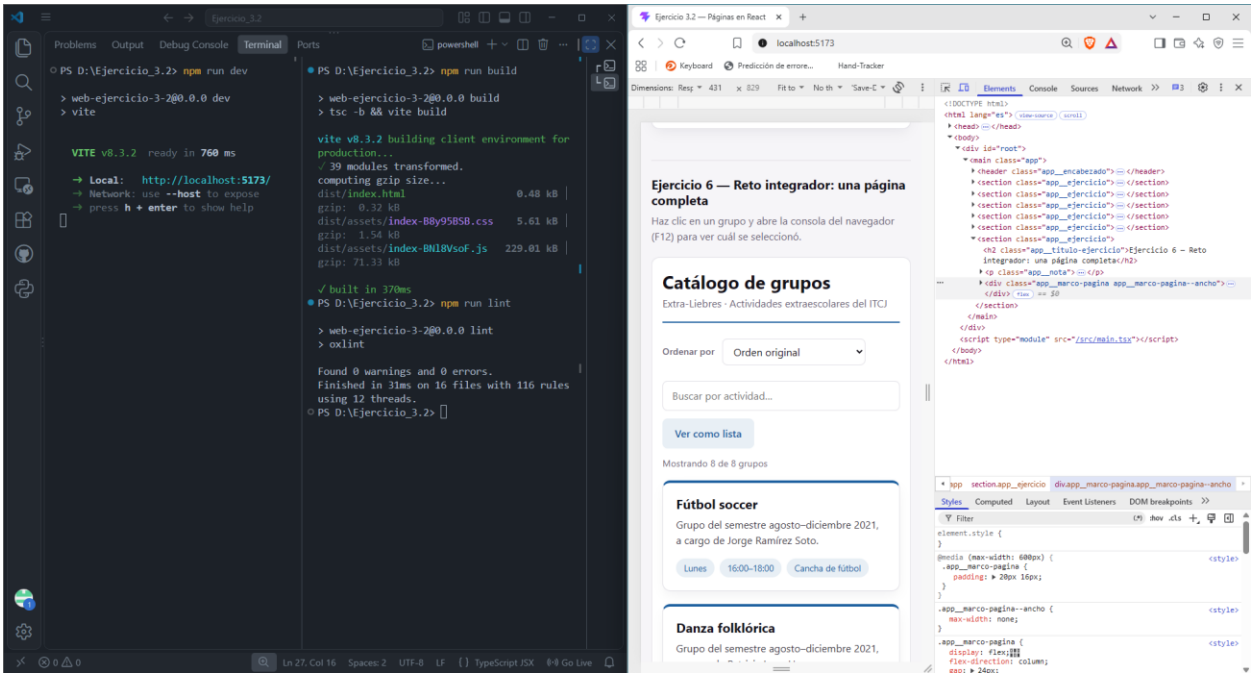


Figura 21. *Compilación y análisis estático sin errores, y vista adaptable.*

Conclusiones

El compendio permitió recorrer los conceptos básicos de React. Primero se escribió un componente con datos fijos; después se volvió reutilizable con props tipadas, se combinó con otros por composición, se usó para mostrar colecciones con map() y key, y finalmente se le agregó estado para que la página respondiera al usuario. El Ejercicio 6 mostró que esos conceptos se combinan sin reescribir nada, pues la página de catálogo se armó casi por completo con componentes de los ejercicios anteriores.

Se reforzaron tres ideas que serán importantes en el proyecto integrador. La primera es que el estado debe vivir en el componente más cercano que contenga a todos los que lo usan. La segunda es que los datos se transforman creando arreglos nuevos en lugar de modificar el original. La tercera es que una función pasada como prop permite que un componente hijo le avise a su padre lo que ocurrió.

Los componentes de los ejercicios 4 a 6 ya trabajan con datos de Extra-Liebres, por lo que son una primera versión de la interfaz del proyecto.